CBS3002 · INFORMATION SECURITY — COURSE PROJECT

SentinelAI

AI-Powered Adaptive Access Control & Insider Threat Detection

TEAM MEMBERS

Atul Bharadwaj — 24BBS0007

Mayank Nitin Sarode — 24BBS0096

Sai Lokesh — 24BBS0114

Hybrid Policy Engine · Behavioral Analytics · Adaptive Security

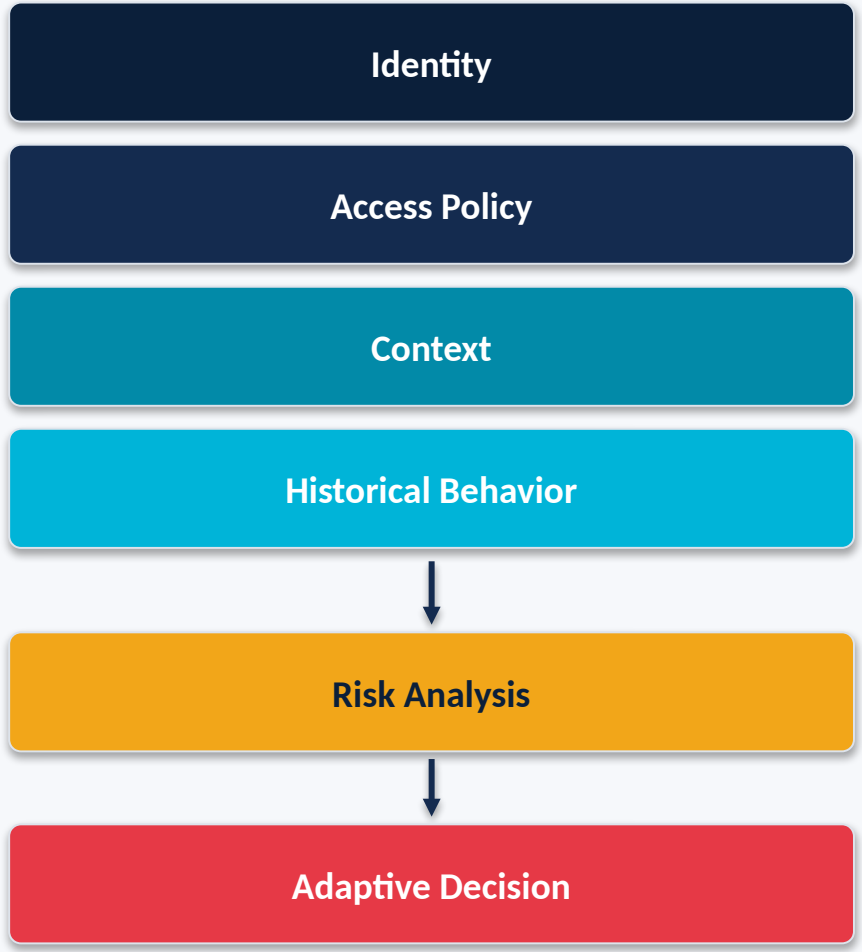
What is SentinelAI?

An intelligent enterprise security platform that combines hybrid access-control policies with machine-learning behavioral anomaly detection to identify suspicious activity from authorized or compromised users.

> *"Authorization determines whether access is permitted by policy;*
> ***behavioral risk determines whether the activity requires additional controls."***

SentinelAI COMBINES:

- Authentication & identity management
- Hybrid access control (RBAC + MAC + ABAC + TBAC)
- Behavioral monitoring & centralized auditing
- ML-based anomaly detection → risk scoring
- Adaptive security responses (Allow / MFA / Block)



Why We Chose This Topic

1

Real-world relevance

Credential compromise & insider misuse are leading causes of breaches; static RBAC alone cannot catch them.

2

Gap in traditional systems

Classic RBAC/MAC/ABAC answer “is this allowed?”, never “is this behavior safe right now?”

3

Breadth of CBS3002 syllabus

Lets us apply access-control models, hybrid policies, auditing, IDS and DB security in one integrated system.

4

Practical ML application

A genuine, well-scoped use case for anomaly detection rather than ML bolted onto a CRUD app.

5

Resume-worthy & extensible

Combines backend engineering, security architecture, and applied ML — a strong, demo-able portfolio project.

Problem Statement

Static authorization (RBAC / MAC / ABAC) only verifies that a user holds valid, permitted credentials — it cannot tell whether the current activity of that account is actually safe.

CORE PROBLEM

- A legitimate account can be compromised (stolen credentials).
- An authorized employee may abuse legitimate privileges.
- Valid credentials + valid role = requests keep getting ALLOWED even when behavior is highly anomalous.
- No behavioral context, no risk signal, no adaptive response → damage (data exfiltration, privilege abuse) goes undetected until too late.

Example: an “allowed” account, 7 minutes

02:13	Login
02:14	Access 35 confidential files
02:16	Download 18 files
02:18	Attempt restricted HR access
02:20	Attempt database access

Static system: credentials valid → ALLOW

What We Are Trying to Achieve

PRIMARY OBJECTIVES

- Secure authentication & authorization framework
- Multiple access-control models (RBAC, MAC, ABAC, TBAC)
- A hybrid policy engine combining all four
- Centralized, tamper-aware security audit logging
- Model normal user behavior from access patterns
- Detect anomalies with machine learning
- Convert anomaly signal → quantified risk score
- Trigger adaptive responses to high-risk activity
- Give admins understandable, AI-assisted event explanations

SECONDARY OBJECTIVES

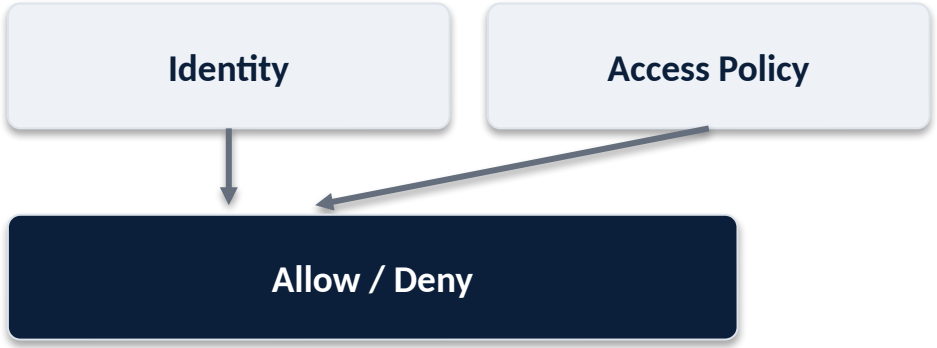
- Demonstrate practical, applied information-security concepts
- Provide measurable ML performance (precision / recall / F1)
- Create realistic, controlled security attack scenarios
- Produce an extensible architecture for future security features

Delivered: the full "Resume-Ready Version"

✓ Hybrid access control (RBAC+MAC+ABAC+TBAC+OBAC) + audit logging
✓ ML anomaly detection (Isolation Forest) + blended risk scoring
✓ Adaptive responses (MFA / email / lockout) + React admin dashboard
✓ AI incident explanations + full Docker deployment

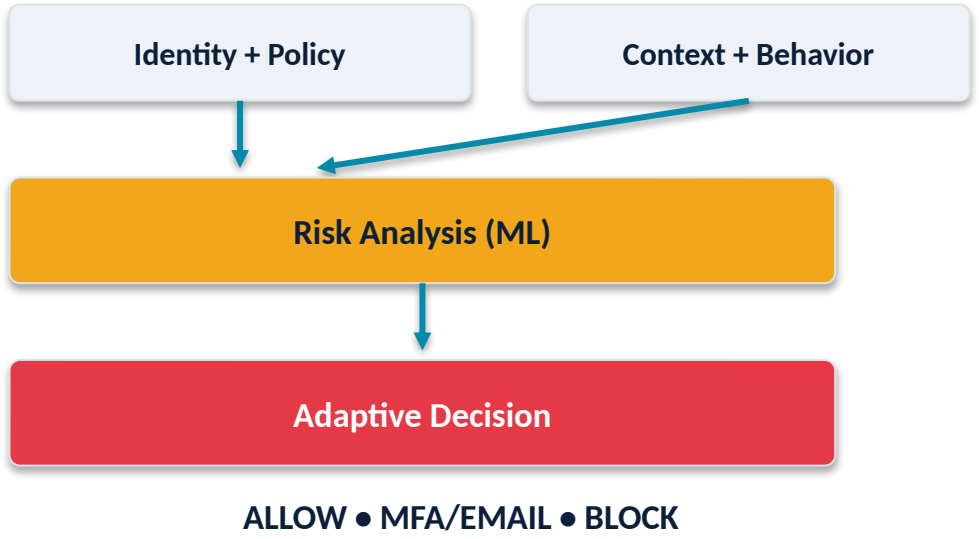
Beyond Static Authorization

Traditional Access Control



Once credentials + role/attributes check out, access is granted — regardless of how the account behaves afterward.

SentinelAI Approach

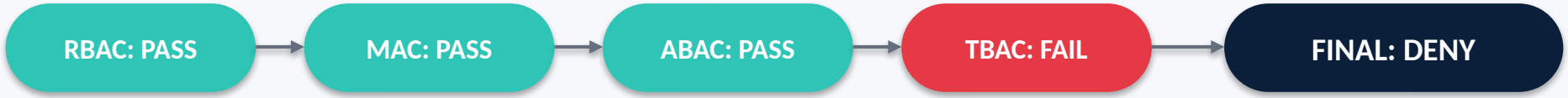


Same valid credentials, but abnormal behavior escalates the response — repeat offenders get locked out for admin review.

Hybrid Policy Engine: RBAC + MAC + ABAC + TBAC + OBAC

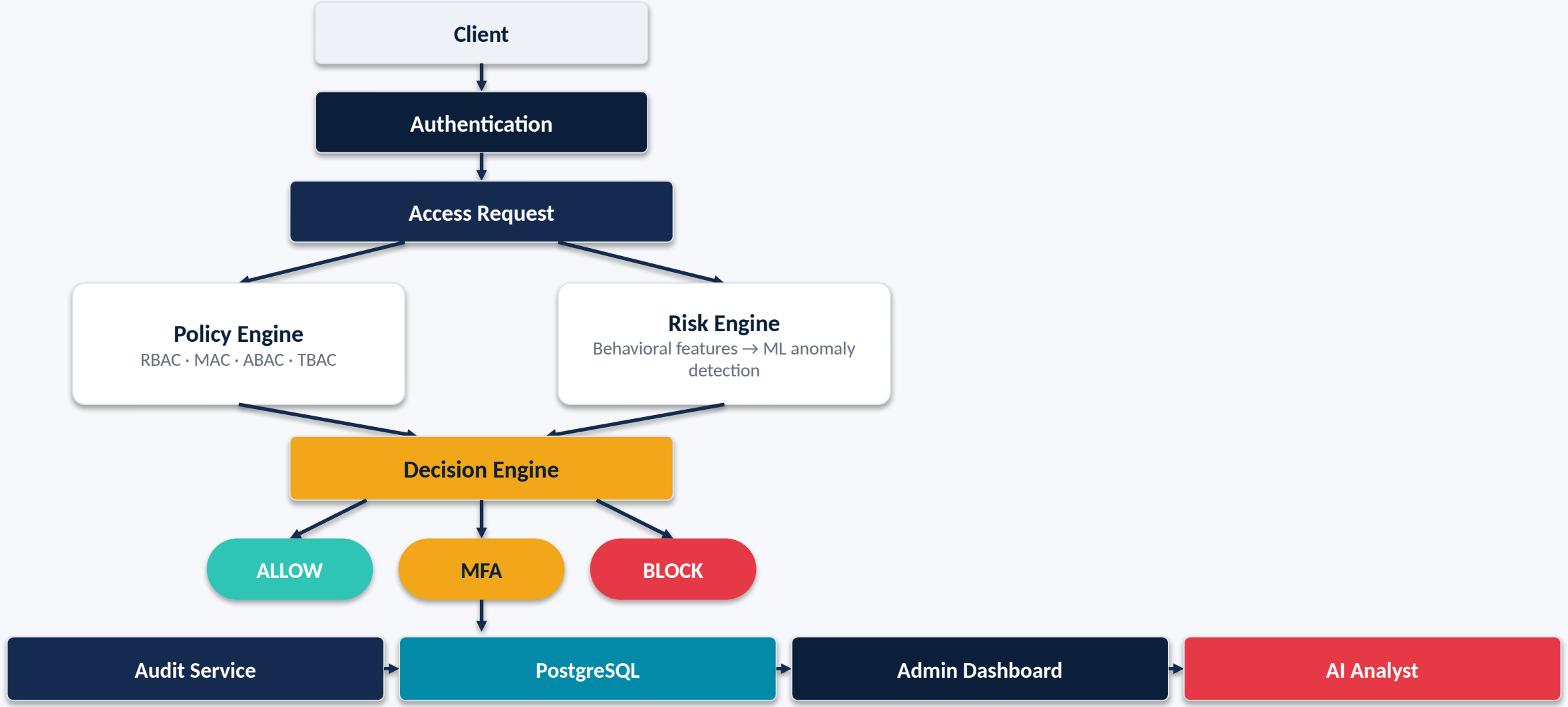
RBAC Role-Based	MAC Mandatory	ABAC Attribute-Based	TBAC Time-Based
Access depends on the user's role (Finance → Financial Reports)	Resource classification vs. user clearance (PUBLIC < CONFIDENTIAL < SECRET < TOP_SECRET)	Department match + trusted device required for access	Access allowed only within a defined time window (e.g. 09:00–18:00)

COMPOSITION RULE: AND (all must pass)



A single failing dimension overrides all passing ones — e.g. an otherwise fully-authorized request outside business hours is still denied by TBAC. A 5th, opt-in layer (OBAC) adds per-action rules on top — e.g. only Admins can DELETE — without ever loosening the first four.

System Architecture



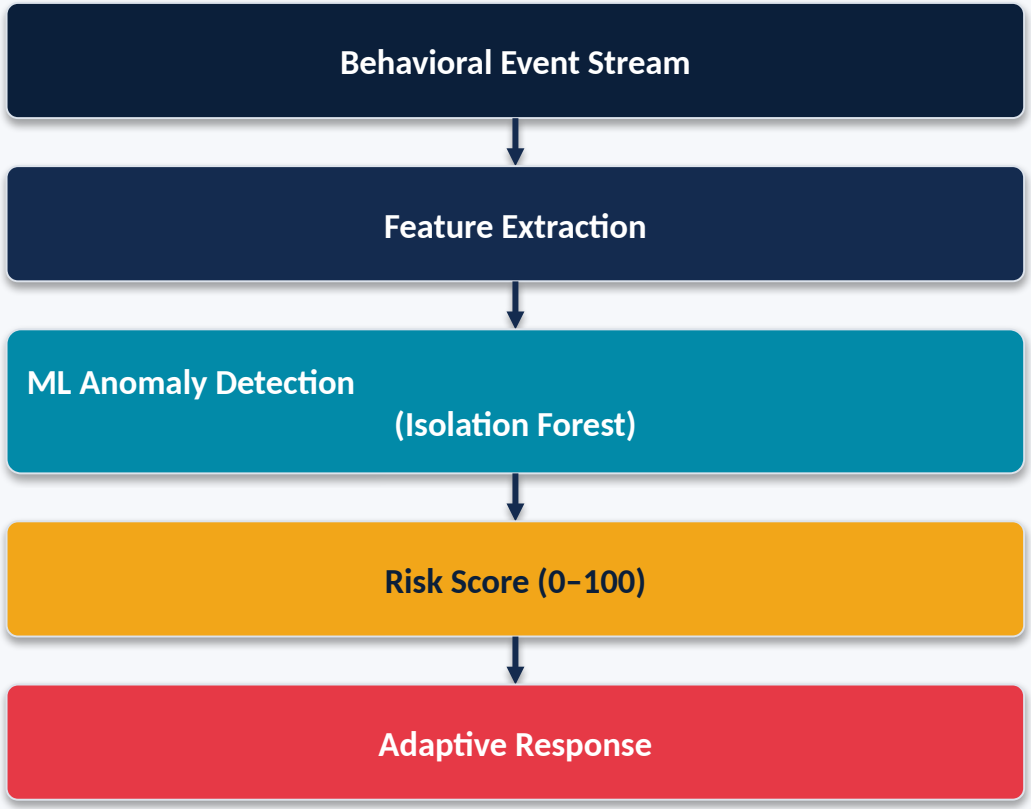
Risk Scoring & Adaptive Security

BEHAVIORAL FEATURES

- Login hour & failed-login count
- Number of resources accessed / downloads
- Request frequency & resource sensitivity
- Location & device familiarity
- Access outside normal hours
- Number of denied requests

RISK SCORE RESPONSE POLICY

0–30	Allow and log
31–60	Allow, increase monitoring
61–80	Require step-up: MFA, else emailed code
81–100	Block + security alert



Authorization stays deterministic; ML only adjusts the strength of the response.

Development Phases

1	**Foundation** Architecture, PostgreSQL, auth, user & resource management	DONE
2	**Access Control** RBAC, MAC, ABAC, TBAC, hybrid policy engine	DONE
3	**Security Monitoring** Centralized audit log, SecurityAlert lifecycle, admin alert queue, 5 encoded attack scenarios	DONE
4	**Machine Learning** Synthetic dataset, 9-feature extraction, trained Isolation Forest, full precision/recall/F1/ROC-AUC evaluation	DONE
5	**Adaptive Security** Blended risk engine, graduated step-up (TOTP MFA or emailed code), auto-block, repeated-abuse lockout	DONE
6	**AI Analyst** Anthropic-powered alert explanations, deterministic template fallback, strictly advisory	DONE
7	**Finalization** React admin console, full Docker Compose stack, architecture/security/threat-model docs, 81 tests	DONE

Final Status

IMPLEMENTED (All 7 Phases)

- ✓ FastAPI backend, 8 modular routers (auth, users, resources, policies, risk, alerts, ai, audit)
- ✓ JWT auth, bcrypt hashing, account lockout, admin reactivation workflow
- ✓ Hybrid Policy Engine: RBAC + MAC + ABAC + TBAC + OBAC + admin override grants
- ✓ Trained Isolation Forest blended with rule-based scoring into a 0–100 risk score
- ✓ Adaptive response: step-up MFA/email verification, auto-block, repeated-abuse lockout
- ✓ Centralized audit log, SecurityAlert lifecycle, AI-assisted incident explanations
- ✓ 81 automated tests incl. 5 encoded attack scenarios — all passing
- ✓ React admin console + full Docker Compose stack (Postgres + FastAPI + React)

COMPLETION BY PHASE



Overall project completion: 100%

All seven phases delivered — hybrid policy engine, ML risk scoring, adaptive response, AI analyst, admin console

Testing: Simulated Attack Scenarios

Each scenario from the threat model is encoded as an automated pytest case against a live FastAPI test client.

#	Scenario	Simulated Behavior	Expected Outcome	Status
1	Brute Force	Repeated failed logins against one account	Account LOCKED + all attempts audited	PASS
2	Credential Compromise	Unusual login time + unknown device on valid credentials	Flagged despite valid credentials (risk layer)	PASS
3	Privilege Abuse	Authorized user requests a role-restricted resource	DENIED by RBAC	PASS
4	Policy Violation	User clearance below resource classification	DENIED by MAC	PASS
5	Mass Download / Exfiltration	High-frequency downloads across many resources in a short window	Risk score spikes → BLOCK + CRITICAL alert	PASS

81 automated tests across auth, policy engine, risk/adaptive security, resource access, audit and alert modules — all passing.

Tools & Technology Used

Backend

- Python 3
- FastAPI
- SQLAlchemy (ORM)
- Pydantic / pydantic-settings

Security

- bcrypt (password hashing)
- PyJWT (access tokens)
- OAuth2 password bearer flow
- pyotp (TOTP) + smtplib (email step-up)

Database

- PostgreSQL 16
- Raw SQL schema (schema.sql)

Machine Learning

- Pandas / NumPy
- Scikit-learn
- Isolation Forest
- Anthropic API (AI analyst)

Frontend

- React
- Vite (dev server & build)
- Fetch-based API client

Testing & DevOps

- Pytest + httpx (81 tests)
- Docker Compose (db + backend + frontend)
- Git / GitHub

Edge Cases We Handled

A few deliberate design decisions that matter more than they look — each one backed by a test in the suite.

MFA Not Configured	Step-up falls back to a one-time emailed code instead of leaving the user stuck — /auth/email-verify/verify
Repeated Abuse	Enough step-up challenges or blocks in a row auto-locks the account — same mechanism as login brute-force, pending admin reactivation
Policy DENY vs. Risk Score	A policy DENY can never be overridden by a low risk score — authorization and behavioral risk are deliberately separate checks
Admin Override Grants	A ResourceGrant can bypass RBAC/MAC/ABAC/TBAC/OBAC for one user, but never the risk engine — a granted user acting suspiciously can still be blocked
OBAC Is Additive, Never Loosening	Opt-in per (resource, action); no matching rule passes trivially — OBAC only adds restrictions on top of the other four
Password-Hashing Edge Case	Passwords are explicitly capped at 72 bytes before bcrypt hashing — bcrypt silently truncates beyond that, so we reject rather than risk a hash collision
Missing ML Model	If the trained Isolation Forest artifact is absent, the risk engine falls back cleanly to rule-based scoring only — no crash

Why This Matters

Who Needs This

- SOC analysts triaging alerts
- Enterprise IT managing access sprawl
- Compliance teams needing an audit trail

What It Stops

- Stolen-credential logins at odd hours
- Silent privilege abuse by authorized staff
- Mass data exfiltration before anyone notices

Beyond This Submission

- Continuous authentication
- Device fingerprinting
- Explainable ML for analyst trust
- SOC/SIEM integration (Splunk, ELK)

Guiding principle throughout: keep authorization deterministic and auditable — ML and AI only ever inform the strength of the response, never replace the policy decision.

CLOSING

A Security Platform Beyond Static Authorization

"A user having permission does not necessarily mean their current behavior is safe."

Identity + Policy + Context + Behavior + ML → Risk Analysis → Adaptive Response.

MAPS DIRECTLY TO CBS3002

- Access control models → RBAC, MAC, ABAC, TBAC
- Hybrid policies & composition → Policy Engine
- Auditing → Centralized audit logging
- Intrusion detection → Behavioral anomaly detection
- Database security → PostgreSQL schema & access control
- Applied ML → Isolation Forest, precision/recall/F1 evaluated

Team

Atul Bharadwaj — 24BBS0007

Mayank Nitin Sarode — 24BBS0096

Sai Lokesh — 24BBS0114

Thank you — Questions?